

Classification d'erreurs par NLP

Comment la plateforme reconnaît le TYPE d'une erreur — expliqué depuis ton code

Plateforme d'apprentissage adaptatif · Yassine Ben Nessib · EspritTech — expliqué à partir du vrai code source

En une phrase — Un classifieur entraîné lit un court texte décrivant une erreur et prédit **de quel type d'erreur (parmi 11)** il s'agit, avec une confiance. S'il n'est pas sûr, il répond « unknown » plutôt que de deviner. Code :

```
error_classification_service.py (usage) + train_error_classifier.py (entraînement).
```

1 L'idée en une minute

Savoir qu'une réponse est fausse ne suffit pas — on veut savoir **pourquoi** : une erreur de signe, une erreur de calcul, une mauvaise méthode... Chacune appelle un indice différent. On entraîne donc un modèle à ranger une erreur dans l'une de **11 catégories** (la taxonomie de `error_taxonomy.py`).

2 Où ça vit dans le code

`app/services/error_classification_service.py`

Utilise le modèle entraîné pour étiqueter une erreur.

`scripts/train_error_classifier.py`

Entraîne le modèle et le sauvegarde.

`app/data/error_classifier.joblib`

Le modèle entraîné sauvegardé (le "cerveau").

`app/data/error_taxonomy.py`

Les 11 types d'erreur (les étiquettes).

3 Entraînement : transformer le texte en nombres (features)

Un modèle ne lit pas des mots, seulement des nombres. On transforme donc chaque texte en **vecteur de features**. Ton code essaie d'abord la voie intelligente (embeddings) et retombe sur la voie classique (TF-IDF).

TRAIN_ERROR_CLASSIFIER.PY — BUILD_FEATURES()

```
def build_features(texts):
    try:
        from app.services import embedding_service
        if embedding_service.is_available():
            X = np.vstack([embedding_service.encode(t) for t in texts])
            return X, "embeddings", None          # PRIMARY: semantic vectors
    except Exception:
        pass
    vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2, lowercase=True)
    X = vec.fit_transform(texts)                  # FALLBACK: TF-IDF
    return X, "tfidf", vec
```

Ce que ça fait, étape par étape :

- **Voie principale** : si les embeddings sont disponibles (Brique 2), chaque texte devient un vecteur de sens. `np.vstack` les empile en une matrice `X` (une ligne par exemple).
- **Repli** : sinon, `TfidfVectorizer` transforme le texte en nombres avec le **TF-IDF** — il pondère chaque mot / paire de mots selon son utilité (fréquent ici mais rare ailleurs = important).
- Elle renvoie `X` (les nombres), la `method` utilisée, et le vectoriseur (sauvegardé pour transformer les nouveaux textes de la même façon).

4 Entraîner le classifieur

TRAIN_ERROR_CLASSIFIER.PY — ENTRAÎNER & SAUVEGARDER

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
clf = LogisticRegression(max_iter=2000, C=4.0)
clf.fit(X_train, y_train)                  # learn from the training part
acc = accuracy_score(y_test, clf.predict(X_test)) # check on unseen part
...
bundle = {"method": method, "vectorizer": vec, "clf": clf_full,
          "classes": list(clf_full.classes_), "threshold": 0.30}
joblib.dump(bundle, OUT)                  # save the trained model
```

Ce que ça fait, étape par étape :

- `train_test_split` coupe les données : 80 % pour **apprendre**, 20 % gardés à part pour **tester** honnêtement. `stratify=y` garde l'équilibre des classes dans les deux parts.
- `LogisticRegression` est un classifieur simple, rapide et explicable — le standard pour le texte. `clf.fit(...)` est l'endroit où il **apprend** vraiment.
- `accuracy_score` mesure à quelle fréquence il a raison sur le jeu de test non vu (plus un rapport par classe et une validation croisée à 5 plis).
- Enfin, tout (méthode, vectoriseur, classifieur, étiquettes, seuil) est rangé dans un `bundle` et sauvegardé avec `joblib.dump` dans le fichier `.joblib`.

5 Utiliser le modèle à l'exécution

ERROR_CLASSIFICATION_SERVICE.PY — CLASSIFY()

```
def classify(text, lang="en"):
    bundle = _load()
    if bundle["method"] == "embeddings":
        vec = embedding_service.encode(text)
        X = np.asarray(vec).reshape(1, -1)      # same features as training
    else:
        X = bundle["vectorizer"].transform([text])
    proba = bundle["clf"].predict_proba(X)[0]   # probability for each class
    idx = int(np.argmax(proba))                 # most likely class
    confidence = float(proba[idx])
    code = bundle["classes"][idx]
    if confidence < bundle["threshold"]:        # not sure enough?
        return {"code": "unknown", "confidence": confidence}
    return {"code": code, "confidence": confidence}
```

Ce que ça fait, étape par étape :

- Elle vectorise le nouveau texte **exactement comme à l'entraînement** (embeddings ou TF-IDF) — c'est crucial, le modèle ne comprend que le même type de nombres.
- `predict_proba` donne une probabilité pour **chacun** des 11 types d'erreur.
- `np.argmax` prend l'indice de la plus grande probabilité ; `classes[idx]` le retransforme en étiquette lisible.
- **Filet de sécurité** : si la probabilité maximale est sous le seuil (0.30), elle renvoie `"unknown"` au lieu de deviner à tort.

POIDS TF-IDF D'UN MOT (POURQUOI UN MOT EST "IMPORTANT")

$$\text{tfidf}(w, d) = \text{tf}(w, d) \times \log(N / \text{df}(w))$$
 tf = nb de fois où w apparaît dans ce texte ·
 N = nb de textes · df = textes contenant w

6 Exemple concret

D'une mauvaise réponse à un indice précis

Un étudiant obtient la bonne valeur mais le **signe opposé**. `classify()` renvoie `{"code": "sign_error", "confidence": 0.82}`. La plateforme affiche un indice ciblé : « vérifie le signe en appliquant les bornes », au lieu d'un vague « incorrect ».

7 Forces & limites

Forces

- Transforme « faux » en diagnostic précis
- Rapide et peu coûteux
- Explicable
- Le filet « unknown » évite les mauvaises devinettes

Limites

- Nécessite des exemples étiquetés
- Ne connaît que les types d'erreur appris
- La qualité dépend des étiquettes
- De nouvelles erreurs imposent un réentraînement

Note honnête — Le modèle a été entraîné sur un jeu étiqueté **synthétique** (exemples générés par type d'erreur via `generate_error_data.py`). L'étape honnête suivante : étiqueter et entraîner sur de **vraies** réponses d'étudiants et reporter la précision de test.

8 Q&R

QUESTIONS DE SOUTENANCE / ENTRETIEN (DONT « CE BLOC FAIT QUOI ? »)

Q Que fait `classify()`, ligne par ligne ?

R Elle charge le modèle, vectorise le texte comme à l'entraînement, obtient une probabilité par classe, prend la plus probable, et — si assez sûre — renvoie ce type d'erreur ; sinon « unknown ».

Q Différence `predict_proba` vs `predict` ?

R `predict` ne donne que la classe choisie ; `predict_proba` donne une probabilité pour chaque classe, ce qui permet de mesurer la confiance et d'appliquer un seuil.

Q Pourquoi un seuil et une classe « unknown » ?

R Pour éviter des étiquettes fausses mais confiantes. Si aucune classe n'est nettement probable, dire « unknown » et retomber sur le comportement par défaut est plus sûr qu'un mauvais indice.

Q Qu'est-ce que le TF-IDF et pourquoi l'utiliser ?

R Une façon de transformer le texte en nombres en pondérant les mots informatifs. C'est le repli classique et léger quand les embeddings ne sont pas disponibles.

Q Pourquoi une régression logistique ?

R Simple, rapide, interprétable, et performante sur de petits jeux de texte avec de bonnes features — le standard, facile à défendre.

Q Comment l'évalues-tu ?

R `train_test_split` garde 20 % non vus ; on reporte l'exactitude, un rapport par classe, et une validation croisée à 5 plis.

9 Termes clés

Vecteur de features — Les nombres qui représentent un texte pour le modèle.

TF-IDF — Pondération des mots qui transforme le texte en nombres.

train_test_split — Couper les données en part d'apprentissage et part de test honnête.

predict_proba — Probabilité donnée par le modèle à chaque classe.

argmax — L'indice de la plus grande valeur.

Seuil — Confiance minimale pour accepter une prédiction.

joblib — Librairie pour sauver/charger un modèle entraîné.

Régression logistique — Un classifieur simple et explicable.